

Intelligent Agents - Complete Notes

1. Four Approaches to AI

Approach	Focus	Description
Think Humanly	Cognitive Science	How humans think - study human reasoning
Act Humanly	Turing Test	Behave like humans - pass as human
Think Rationally	Logic	Think logically - use formal logic
Act Rationally	Rational Agents	Act optimally - maximize expected performance

Most Important: Act Rationally - This is what we focus on in AI agents.

1.1 Weak AI vs Strong AI

Type	Description	Examples
Weak AI (Narrow AI)	AI designed for specific tasks	Chess programs, Siri, Google Translate
Strong AI (General AI)	AI with human-level intelligence across all domains	Hypothetical - doesn't exist yet

- **Current AI:** All existing AI systems are Weak/Narrow AI
- **Future Goal:** Strong AI that can think, learn, and reason like humans

2. What is an Agent?

- **Definition:** Anything that perceives its environment through sensors and acts through actuators
- **Formula:** Agent = Architecture + Program
- **Agent Function:** Maps percept histories to actions [f: $P^* \rightarrow A$]

Types of Agents:

- **Human:** Eyes/ears (sensors), hands/legs (actuators)
- **Robot:** Cameras (sensors), motors (actuators)
- **Software:** Programs that act on behalf of users

3. Rational Agents

- **Goal:** Do the "right thing" to be most successful

- **Performance Measure:** Objective criterion for success
- **Rational Agent:** Selects actions that maximize expected performance
- **Key Points:**
 - Rationality $\neq$ Omniscience (all-knowing)
 - Can gather information and learn
 - Autonomous agents learn from experience

4. PEAS Framework

When designing agents, consider:

- **Performance measure:** How success is measured
- **Environment:** Where the agent operates
- **Actuators:** How the agent acts
- **Sensors:** How the agent perceives

Example - Taxi Driver:

- **Performance:** Safe, fast, legal, profitable trips
- **Environment:** Roads, traffic, pedestrians
- **Actuators:** Steering, brakes, horn
- **Sensors:** Cameras, GPS, speedometer

5. Environment Types

Dimension	Type 1	Type 2
Observability	Fully Observable	Partially Observable
Predictability	Deterministic	Stochastic
Episodes	Episodic	Sequential
Change	Static	Dynamic
States	Discrete	Continuous
Agents	Single Agent	Multi-Agent

Examples:

- **Chess:** Fully observable, deterministic, sequential, static, discrete, multi-agent
- **Taxi Driving:** Partially observable, stochastic, sequential, dynamic, continuous, multi-agent

6. Agent Types (by Intelligence Level)

1. Simple Reflex Agents

- React to current percept only
- "If X happens, do Y"
- Example: Alarm clock

2. Model-Based Reflex Agents

- Consider how world changes
- Track state over time
- Example: Roomba vacuum

3. Goal-Based Agents

- Have specific goals to achieve
- Consider future states
- Example: GPS navigation

4. Utility-Based Agents

- Choose best option among goals
- Optimize preferences
- Example: GPS finding fastest route

5. Learning Agents

- Adapt from feedback and experience
- Components: Learning element, Performance element, Critic, Problem generator
- Example: Human agents

Problem Solving & Search - Complete Notes

7. Problem Formulation

To solve problems using search, we need 6 components:

1. **State:** Current situation/configuration
2. **Initial State:** Starting point

3. **Actions/Successor Function:** Available moves from each state
4. **Goal Test:** Check if current state is the goal
5. **Path Cost:** Cost of reaching current state
6. **Solution:** Sequence of actions from initial to goal state

Example - Romania Map Problem:

- **State:** Cities (Arad, Sibiu, Bucharest, etc.)
- **Initial State:** In(Arad)
- **Actions:** Go(Sibiu), Go(Timisoara), Go(Zerind)
- **Goal Test:** In(Bucharest)?
- **Path Cost:** Sum of distances
- **Solution:** Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

8. Search Strategies Evaluation Criteria

- **Completeness:** Will it find a solution if one exists?
- **Time Complexity:** How long does it take?
- **Space Complexity:** How much memory needed?
- **Optimality:** Will it find the best solution?

Variables: b = branching factor, d = depth of solution, m = max depth

9. Uninformed Search (Blind Search)

Definition: Search strategies that have no additional information beyond the problem definition. They can only distinguish goal states from non-goal states.

9.1 Breadth-First Search (BFS)

- **Strategy:** Explore all nodes at current level before going deeper
- **Data Structure:** FIFO Queue
- **Working:**
 1. Start with initial state in queue
 2. Remove first node from queue
 3. If goal, return solution
 4. Add all children to end of queue
 5. Repeat until queue empty or goal found

- **Properties:**
 - Complete: Yes (if b is finite)
 - Time: $O(b^d)$
 - Space: $O(b^d)$
 - Optimal: Yes (if all step costs equal)

9.2 Uniform-Cost Search (UCS)

- **Strategy:** Expand node with lowest path cost first
- **Data Structure:** Priority Queue (ordered by path cost)
- **Working:**
 1. Start with initial state in priority queue
 2. Remove node with lowest path cost
 3. If goal, return solution
 4. Add children with updated path costs
 5. Repeat until queue empty or goal found
- **Properties:**
 - Complete: Yes (if b finite, positive step costs)
 - Time: $O(b^{(cost^*/\epsilon)})$ where $cost^*$ is optimal cost
 - Space: $O(b^{(cost^*/\epsilon)})$
 - Optimal: Yes

9.3 Depth-First Search (DFS)

- **Strategy:** Go as deep as possible, then backtrack
- **Data Structure:** LIFO Stack
- **Working:**
 1. Start with initial state on stack
 2. Remove top node from stack
 3. If goal, return solution
 4. Add children to top of stack
 5. Repeat until stack empty or goal found
- **Properties:**
 - Complete: No (can get stuck in infinite loops)

- Time: $O(b^m)$
- Space: $O(bm)$
- Optimal: No

9.4 Depth-Limited Search (DLS)

- **Strategy:** DFS with maximum depth limit l
- **Working:** Same as DFS but stop expanding at depth l
- **Properties:**
 - Complete: No (if $l < d$)
 - Time: $O(b^l)$
 - Space: $O(bl)$
 - Optimal: No

9.5 Iterative Deepening Search (IDS)

- **Strategy:** Repeatedly apply DLS with increasing depth limits
- **Working:**
 1. Try DLS with limit 0, then 1, then 2, etc.
 2. Stop when solution found or no more nodes
- **Properties:**
 - Complete: Yes (if b is finite)
 - Time: $O(b^d)$
 - Space: $O(bd)$
 - Optimal: Yes (if step cost = 1)

9.6 Bidirectional Search

- **Strategy:** Search simultaneously from start and goal
 - **Working:** Run two searches and stop when they meet
 - **Properties:**
 - Complete: Yes (if both directions are complete)
 - Time: $O(b^{(d/2)})$
 - Space: $O(b^{(d/2)})$
 - Optimal: Yes (if both searches are optimal)
-

10. Informed Search (Heuristic Search)

Definition: Search strategies that use problem-specific knowledge beyond the problem definition. They use heuristic functions to guide search toward goal.

Heuristic Function $h(n)$: Estimated cost from node n to goal

- $h(n) \geq 0$ for all nodes n
- $h(\text{goal}) = 0$

10.1 Greedy Best-First Search

- **Strategy:** Expand node that appears closest to goal
- **Formula:** $f(n) = h(n)$
- **Data Structure:** Priority queue ordered by $h(n)$
- **Working:**
 1. Start with initial state in priority queue
 2. Remove node with lowest $h(n)$ value
 3. If goal, return solution
 4. Add children with their $h(n)$ values
 5. Repeat until goal found or queue empty
- **Properties:**
 - Complete: No (can get stuck in loops)
 - Time: $O(b^m)$ in worst case
 - Space: $O(b^m)$
 - Optimal: No
- **Advantage:** Fast when heuristic is good
- **Disadvantage:** Can be misled by poor heuristic

10.2 A* Search

- **Strategy:** Minimize $f(n) = g(n) + h(n)$
- **Formula:**
 - $f(n) = g(n) + h(n)$
 - $g(n)$ = actual cost from start to n
 - $h(n)$ = heuristic cost from n to goal

- **Data Structure:** Priority queue ordered by $f(n)$
- **Working:**
 1. Start with initial state in priority queue
 2. Remove node with lowest $f(n)$ value
 3. If goal, return solution
 4. Add children with updated $g(n)$ and $f(n)$ values
 5. Repeat until goal found or queue empty
- **Properties** (when $h(n)$ is admissible):
 - Complete: Yes
 - Time: $O(b^d)$ in worst case
 - Space: $O(b^d)$
 - Optimal: Yes
- **Admissible Heuristic:** $h(n) \leq \text{actual cost to goal}$
- **Consistent Heuristic:** $h(n) \leq c(n, n') + h(n')$ for every successor n'

10.3 Memory-Bounded A* (SMA*)

- **Strategy:** A* with limited memory
- **Working:**
 1. Run A* normally until memory limit reached
 2. Delete worst leaf node (highest f-value)
 3. Back up f-value to parent before deleting
 4. Continue search with available memory
- **Properties:**
 - Complete: Yes (if solution fits in memory)
 - Time: Depends on memory available
 - Space: Limited to available memory
 - Optimal: Yes (if optimal solution fits in memory)
- **Advantage:** Uses available memory optimally
- **Disadvantage:** May need to regenerate forgotten nodes

10.4 Recursive Best-First Search (RBFS)

- **Strategy:** Recursive A* with f-limit

- **Working:**
 1. Expand best node until f-value exceeds limit
 2. Back up best f-value to parent
 3. Recursively search other branches when limit exceeded
 - **Properties:**
 - Complete: Yes (with admissible heuristic)
 - Time: Depends on accuracy of heuristic
 - Space: $O(bd)$
 - Optimal: Yes (with admissible heuristic)
 - **Advantage:** Linear space complexity
 - **Disadvantage:** May regenerate nodes multiple times
-

11. Local Search and Optimization

Definition: Search algorithms that operate on single current state rather than maintaining paths. Used for optimization problems where path doesn't matter, only the final state.

Characteristics:

- Use constant space
- Find reasonably good solutions in large/infinite state spaces
- Suitable for optimization problems
- Don't maintain search trees

11.1 Hill Climbing (Steepest-Ascent)

- **Strategy:** Continuously move to better neighboring state
- **Working:**
 1. Start with random initial state
 2. Evaluate all neighbors
 3. Move to best neighbor if better than current
 4. Repeat until no better neighbor exists
- **Properties:**
 - Complete: No
 - Time: Depends on landscape

- Space: $O(1)$
- Optimal: No
- **Problems:**
 - **Local Maxima:** Peaks that aren't global maximum
 - **Ridges:** Sequences of local maxima
 - **Plateaus:** Flat areas with no improvement
- **Variants:**
 - **Stochastic Hill Climbing:** Choose randomly among uphill moves
 - **First-Choice Hill Climbing:** Choose first uphill move
 - **Random-Restart Hill Climbing:** Multiple random starts

11.2 Local Beam Search

- **Strategy:** Keep track of k states instead of just one
- **Working:**
 1. Start with k randomly generated states
 2. Generate all successors of k states
 3. Select k best successors
 4. Repeat until goal found
- **Properties:**
 - Complete: No
 - Time: Depends on k and landscape
 - Space: $O(k)$
 - Optimal: No
- **Advantage:** More exploration than hill climbing
- **Problem:** Can concentrate on small region
- **Solution: Stochastic Beam Search** - choose k successors randomly

11.3 Simulated Annealing

- **Strategy:** Allow "bad" moves with decreasing probability
- **Working:**
 1. Start with initial state and high temperature T
 2. Pick random successor

3. If better, accept move
 4. If worse, accept with probability $e^{(\Delta E/T)}$
 5. Decrease temperature T over time
 6. Repeat until T approaches 0
- **Formula:** $P(\text{accept bad move}) = e^{(\Delta E/T)}$
 - ΔE = change in evaluation function
 - T = temperature parameter
 - **Properties:**
 - Complete: Yes (with proper cooling schedule)
 - Time: Depends on cooling schedule
 - Space: $O(1)$
 - Optimal: Yes (in limit, with slow cooling)
 - **Cooling Schedule:** How T decreases over time
 - **Advantage:** Can escape local optima
 - **Disadvantage:** Requires tuning of temperature schedule

11.4 Genetic Algorithm

- **Strategy:** Evolutionary approach using population of states
- **Working:**
 1. Start with population of random states (individuals)
 2. Evaluate fitness of each individual
 3. Select pairs of individuals for reproduction
 4. Apply crossover and mutation operators
 5. Replace population with new generation
 6. Repeat until satisfactory solution found
- **Key Components:**
 - **Individual:** Single state/solution
 - **Population:** Set of individuals
 - **Fitness Function:** Evaluation measure
 - **Selection:** Choose parents for reproduction
 - **Crossover:** Combine two parents to create offspring

- **Mutation:** Random changes to individuals
 - **Properties:**
 - Complete: No
 - Time: Depends on population size and generations
 - Space: $O(\text{population size})$
 - Optimal: No guarantee
 - **Selection Methods:**
 - **Roulette Wheel:** Probability proportional to fitness
 - **Tournament:** Random groups compete
 - **Rank-Based:** Based on fitness ranking
 - **Advantages:**
 - Explores multiple areas simultaneously
 - Good for complex search spaces
 - Naturally parallel
 - **Disadvantages:**
 - Many parameters to tune
 - No guarantee of optimality
 - Can be slow to converge
-

Most Important Questions with Answers

Agent-Related Questions

1. What are the four approaches to AI?

Answer:

- **Think Humanly:** Study how humans think (Cognitive Science approach)
- **Act Humanly:** Make systems behave like humans (Turing Test approach)
- **Think Rationally:** Use formal logic and reasoning (Logic-based approach)
- **Act Rationally:** Act to achieve best expected outcome (Rational Agent approach)

2. What is an intelligent agent?

Answer: An agent is anything that perceives its environment through sensors and acts upon that environment through actuators to achieve its goals.

3. Define rational agent.

Answer: A rational agent selects actions that maximize its expected performance measure, given the percept sequence and built-in knowledge it has.

4. What does PEAS stand for?

Answer:

- **Performance measure:** How success is measured
- **Environment:** Where the agent operates
- **Actuators:** How the agent acts on environment
- **Sensors:** How the agent perceives environment

5. List the five types of intelligent agents.

Answer:

1. Simple Reflex Agents
2. Model-based Reflex Agents
3. Goal-based Agents
4. Utility-based Agents
5. Learning Agents

6. Differentiate between fully observable and partially observable environments.

Answer:

- **Fully Observable:** Agent's sensors can detect all relevant aspects of environment (e.g., Chess)
- **Partially Observable:** Agent cannot see all relevant information (e.g., Card games)

7. What is the difference between deterministic and stochastic environments?

Answer:

- **Deterministic:** Next state completely determined by current state and action (e.g., Chess)
- **Stochastic:** Next state has randomness/uncertainty (e.g., Taxi driving)

8. Define PEAS for an automated taxi driver.

Answer:

- **Performance:** Safe, fast, legal, comfortable trip, maximize profits
- **Environment:** Roads, other traffic, pedestrians, weather
- **Actuators:** Steering wheel, accelerator, brake, signal, horn
- **Sensors:** Cameras, sonar, speedometer, GPS, odometer

9. What are simple reflex agents?

Answer: Agents that select actions based only on current percept, ignoring history. They follow "if-then" rules (e.g., if dirty then suck). Limited intelligence with no memory.

10. What is the difference between weak AI and strong AI?

Answer:

- **Weak AI (Narrow AI):** Designed for specific tasks only (e.g., chess programs, Siri, image recognition)
- **Strong AI (General AI):** Would have human-level intelligence across all domains - currently doesn't exist

Search-Related Questions

11. What are the six components of problem formulation?

Answer:

1. **State:** Current situation/configuration
2. **Initial State:** Starting point
3. **Actions:** Available moves from each state
4. **Goal Test:** Check if current state is goal
5. **Path Cost:** Cost of reaching current state
6. **Solution:** Sequence of actions from initial to goal

12. Compare BFS and DFS.

Answer:

Property	BFS	DFS
Strategy	Level by level	Deep first, then backtrack
Data Structure	Queue (FIFO)	Stack (LIFO)
Complete	Yes	No (infinite loops)
Optimal	Yes (equal costs)	No
Space	$O(b^d)$	$O(bm)$

13. What is Uniform-Cost Search?

Answer: UCS expands the node with lowest path cost first. It's optimal and complete, using a priority queue ordered by path cost. It's like BFS but considers actual costs instead of just depth.

14. Explain Iterative Deepening Search.

Answer: IDS repeatedly applies depth-limited search with increasing depth limits (0, 1, 2, ...). It combines advantages of BFS (optimal, complete) and DFS (low memory). Though it revisits nodes, the overhead is minimal.

15. What is A* search and why is it optimal?

Answer: A* uses $f(n) = g(n) + h(n)$ where $g(n)$ is actual cost and $h(n)$ is heuristic estimate. It's optimal when heuristic is admissible (never overestimates). It combines benefits of UCS and Greedy search.

16. Difference between informed and uninformed search?

Answer:

- **Uninformed:** No additional information beyond problem definition (BFS, DFS, UCS)
- **Informed:** Uses heuristic function to guide search toward goal (A*, Greedy)

17. What is hill climbing and its problems?

Answer: Hill climbing moves to best neighboring state until no improvement possible. Problems: local maxima (trapped at peaks), ridges (sequences of local maxima), plateaus (flat areas with no progress).

18. How does simulated annealing escape local optima?

Answer: Accepts bad moves with probability $e^{-(\Delta E/T)}$. High temperature T allows more bad moves initially, then T decreases over time. This allows escape from local optima early while converging later.

19. What is the difference between rationality and omniscience?

Answer: Rationality means making best decision with available information. Omniscience means knowing everything. Rational agents can't be omniscient but can gather information and learn to improve decisions.

20. When to use local search vs tree search?

Answer: Use local search when: path doesn't matter (only final state), state space is large/infinite, need optimization, memory is limited. Use tree search when: path matters, need systematic exploration, completeness required.